DECA Lab Logbook — Spring Term

Lab 2 | Task 3: Multiply Algorithm Hand Trace

Task 3: Multiply Algorithm Hand Trace

Objectives

- Verify the shift-and-add multiply algorithm by tracing op1 = 12, op2 = 5 by hand
- Relate the algorithm to the binary shift-and-add multiply method from Lecture 3

Task 3. Walk through the code in Figure 1 by hand with op1 = 12, op2 = 5 to check that it works correctly and generates 60 as an answer. Compare its operation with the shift and add multiply method explained in Lecture 3 to show how it works.

Algorithm Principle

The expression op1 & 1 isolates the LSB of op1. For each iteration, bk denotes the k-th bit of op1. The algorithm computes:

$$op1 \times op2 = \Sigma\, bk \cdot (op2 \ll k), \quad \text{for } k = 0 \ldots n-1$$

Each iteration: right-shift op1 by 1 to expose the next bit; left-shift op2shifted by 1 to double its positional value. If the current LSB of op1 is 1, add op2shifted to sum; otherwise add nothing. This is identical to the Lecture 3 shift-and-add method.

Hand Trace: op1 = 12 (0b1100), op2 = 5 (0b0101)

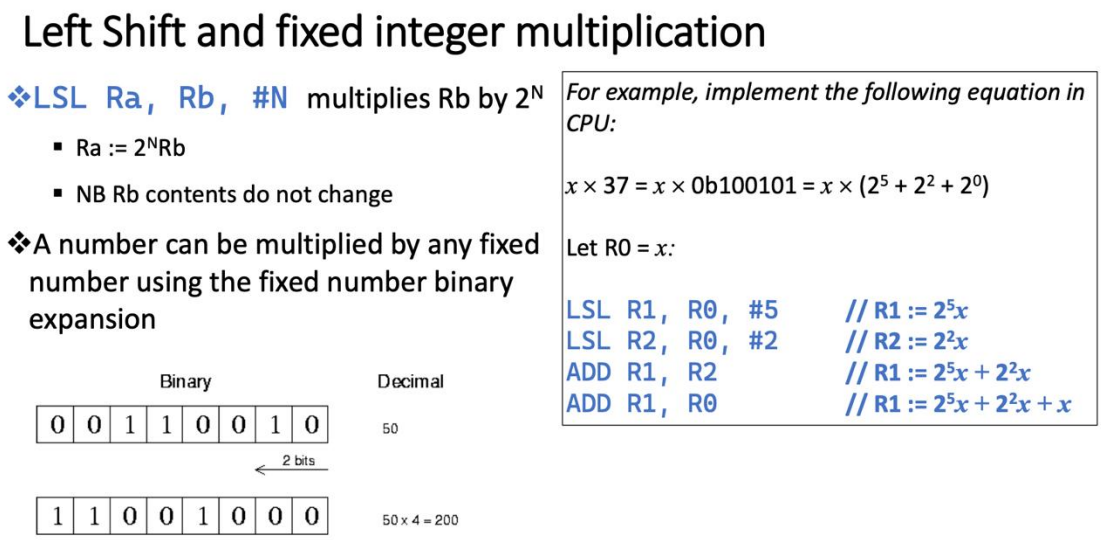
Round	op1 (binary)	op1 & 1	op2shifted	Action on sum	sum
Init	0b1100	—	5	Initialise sum = 0	0
1	0b1100	0	5	LSB = 0 → sum unchanged	0
	0b0110	—	10	op1 >>1 → 0b0110; op2shifted <<1 → 10	
2	0b0110	0	10	LSB = 0 → sum unchanged	0
	0b0011	—	20	op1 >>1 → 0b0011; op2shifted <<1 → 20	
3	0b0011	1	20	LSB = 1 →	20

Round	op1 (binary)	op1 & 1	op2shifted	Action on sum	sum
				sum += 20	
	0b0001	—	40	op1 >>1 → 0b0001; op2shifted <<1 → 40	
4	0b0001	1	40	LSB = 1 → sum += 40	60
	0b0000	—	80	op1 >>1 → 0b0000; loop ends	

After Round 4, op1 = 0, the while loop terminates. Final sum = 60 = 12 × 5. √

Correspondence with Lecture 3 Shift-and-Add Method

The algorithm implements the binary expansion of multiplication directly:

- op1 = 12 = 0b1100 = $2^3 + 2^2$, so 12 × 5 = $(2^3 \times 5) + (2^2 \times 5)$ = 40 + 20 = 60.
- Rounds 1 and 2 correspond to bits 0 and 1 of op1 (both 0) — no addition.
- Rounds 3 and 4 correspond to bits 2 and 3 of op1 (both 1) — op2shifted = 20 and 40 are added.

The right-shift of op1 extracts successive bits without requiring explicit bit addressing. The left-shift of op2shifted keeps it aligned with the positional weight of the current op1 bit, exactly matching the Lecture 3 fixed-shift multiplication framework generalised to variable operands.